THE OPD SAATHI STORY · PATIENT HISTORY-TAKING AND QUEUE SOFTWARE

OPD Saathi

Every patient heard — in their own language — before they reach the doctor.

A multilingual, voice-first check-in kiosk, a fair live queue and a one-page English case sheet for crowded government OPDs. This document tells the whole story: the need, the people it serves, a full end-to-end journey, every feature, the queuing algorithm, security, scale, design and the technology behind each part — with the reason we chose it.

Hindi · Marathi · English | Tap, speak or type | Fair, explainable queue | AI that assists, never decides | Encrypted, per-hospital data | Works without internet | One setup per hospital | History only — no diagnosis

Prepared 11 October 2026 from the code on the main branch (after pull request #9). All patients, staff and hospitals shown are fictional.

IN ONE MINUTE

The pitch

A patient walks into a government OPD. Instead of standing in a line to repeat their story to a busy doctor in a language they may not share, they spend three to five minutes at a kiosk — tapping, speaking or typing in Hindi, Marathi or English. Before they sit down, the doctor already has a clean English case sheet, the nurse can see who needs attention first, and the patient can follow their turn on their own phone.

For patients

Their language, their words, a fair place in line and a live token on their phone. No login, no app, no reading needed.

For doctors and nurses

History collected before the consultation, warning signs surfaced first, and a live queue that explains every position.

For the hospital

Reports on waits, load and languages; staff, kiosk and department management; patient data that never leaves its own server.

What makes it different

IDEA	WHY IT MATTERS
Safety before questions	Six danger signs are asked first. A patient with chest discomfort alerts the nurse within seconds, not after a full interview.
A queue anyone can audit	Every position comes with a "Why?" that lists each point. The sickest go first, but no one waits forever, and typing "10/10" cannot jump the line.
AI as a checker, not a decider	AI translates, summarises and double-checks. Safety rules decide, AI can only add caution, and staff confirm. Everything still works with AI switched off.
Built for real hospitals	Each hospital runs its own copy with its own database; kiosks keep working when the internet does not; data is encrypted and walled off per hospital inside the database itself.
Inclusive by design	Voice in three languages, big buttons, high contrast, call-staff on every screen, and nurse-assisted registration for anyone who cannot use the kiosk.

3–5 min

typical kiosk check-in

8–15 adaptive questions, mostly one tap each (demo history averages 4.7 minutes).

1,000+

automated checks

814 server tests, 54 web tests, 132 browser tests, plus synthetic evaluations — all passing.

0

diagnoses made

By design. OPD Saathi collects history; clinicians decide.

1 · THE NEED

A morning in a government OPD

The registration counter opens and the corridor is already full. Most of the problems below are familiar to anyone who has stood in that corridor.

What patients face

- **Language.** A Marathi-speaking grandmother, a Hindi-speaking labourer and an English-speaking student meet the same doctor. Describing symptoms well in a second language is hard, and details get lost.
- **Literacy.** Many patients cannot fill a form. Paper slips and long questionnaires exclude them.
- **Uncertainty.** Nobody knows their turn or how long the wait is, so everyone crowds the doctor's door and no one can step away.
- **Fairness.** "Why was that person called before me?" — without an answer, trust falls and arguments rise.

What clinicians face

- **Very little time per patient.** Much of it goes on basic questions — since when, how bad, which medicines, any allergies — before real examination starts.
- **Unstructured history.** The story arrives in pieces, in different languages, with nothing written down.
- **Hidden urgency.** A patient with a warning sign can sit quietly in the line behind twenty routine visits.
- **No picture of the day.** Administrators cannot see waiting times, department load or language mix to plan staff.

The problem we set out to solve

Patient history-taking for crowded OPDs: collect a structured patient history before the consultation, in the patient's language, and help staff manage the OPD queue — without diagnosing or recommending treatment.

What a good answer has to do

1. Take history **in the patient's language**, in a way that works for people who cannot read.
2. Catch **danger signs immediately**, not at the end.
3. Hand the doctor a **short, structured English summary** — with the patient's own words a click away.
4. Make the queue **live, fair and explainable**, and tell patients where they stand.
5. Be **safe, private and affordable** for public hospitals: no diagnosis, encrypted data, works on ordinary hardware and without reliable internet.

> **SAY IT ON STAGE**
> The problem is not a lack of doctors' skill — it is the minutes lost to collecting history, the language gap, and a queue nobody can see. OPD Saathi gives those minutes back.

2 · SOCIAL IMPACT

Who we serve, and why it matters

Public OPDs serve the people with the fewest alternatives. Every design decision starts from them.

PERSON	WHAT OPD SAATHI CHANGES FOR THEM
Elderly patients	Large 64-pixel buttons, every screen read aloud, high-contrast mode, a "Call staff" button everywhere, and the 75+ age group counted in the fair queue.
Patients who cannot read	Most answers are one tap on a picture-like choice; they can also just speak. The whole history is read back to them before they confirm it.
Speakers of Hindi or Marathi	The entire kiosk, consent, questions, read-back and status page are in their language. The doctor reads English but can open the patient's original words at any time.
Mothers, infants and pregnant women	Infants, young children and pregnancy add points to the triage score, so they get a fair head start without anyone having to argue for it.
The seriously ill	Danger signs come first and alert staff in real time; urgent patients go to the front of the queue.
Anyone who cannot use a kiosk	A nurse can register them (assisted registration) with the same consent, safety checks and queue.
Doctors and nurses	Less time on routine questions, more on examination and care; a calmer waiting room because patients can see their turn.
Hospital administrators	Evidence for planning — waits, consultation times, department load and language mix — without extra data entry.

Dignity and trust

- The patient tells their story in their own words and confirms it.
- Patients never see a "severity score" or a disease name — only neutral, respectful wording.
- Consent comes first, in plain language, and saying "No, I need staff help" is always possible.

Fit for public health

- Runs on ordinary PCs and tablets inside the hospital; no per-patient cost without AI.
- Patient data stays in the hospital's own building and database.
- Ready for more Indian languages — the whole app is translation-driven.

SAY IT ON STAGE

OPD Saathi is built for the patient who is old, unwell, cannot read, or does not speak the doctor's language — and it treats them with the same dignity and fairness as everyone else in the line.

3 · THE SOLUTION

One system, one patient journey

Kiosk
Language, consent, age/sex, danger signs, 8–15 questions, read-back

→

Token + QR
Department, place in line, live phone status

→

Nurse
Live fair queue, alerts, call next

→

Doctor
English case sheet, transcript, edits, PDF

→

Admin
Reports, staff, kiosks, departments

Patient kiosks
any tablet / PC browser

Patient's phone
QR status page

Staff browsers
laptop, tablet, phone

React web app (PWA)
one app for kiosk, status page and staff · works offline for the short form

FastAPI server
interview engine · safety rules · triage · routing · queue · case sheets · reports · live updates

Background worker (ARQ)
publishes queue changes, refreshes reports, closes old days

PostgreSQL 16
records, encrypted fields, per-hospital row-level security

Redis 7
live queue, sessions, rate limits (+ a separate cache)

Optional AI & voice
Gemini · Sarvam (Saaras, Bulbul)

Everything is one web application served by the hospital's own server. Kiosks, staff screens and patients' phones are just browsers. All of it starts with one command inside Docker.

Five audiences, one source of truth

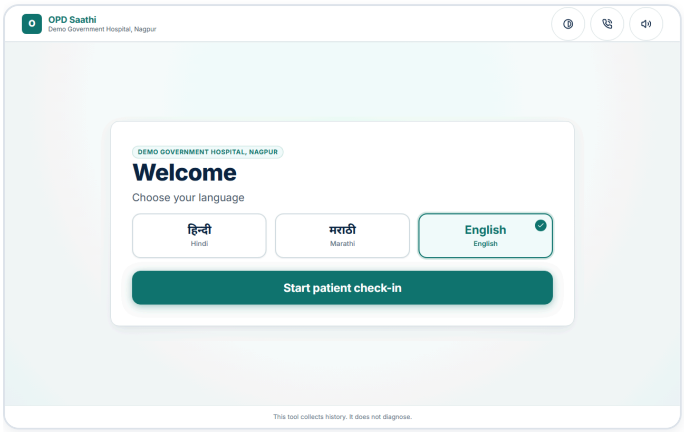
WHO	WHERE	WHAT THEY DO
Patient	Kiosk `/kiosk`	Checks in by tapping, speaking or typing; gets a token and a QR code.
Patient's phone	QR → `/s/<token>`	Live place in line, estimated wait, Waiting → Called → With doctor → Done.
Nurse	Queue, Patients	Runs the live queue, answers alerts, calls patients, registers those who need help.
Doctor	Queue, Consultations	Reads and corrects the case sheet, starts and finishes consultations, prints a PDF.
Administrator	Reports, Settings	Sees how the OPD is doing; manages staff, kiosks and departments.

> **SAY IT ON STAGE**
> One React app, one Python server, one database per hospital — the kiosk, the nurse's queue, the doctor's case sheet and the patient's phone all see the same live truth.

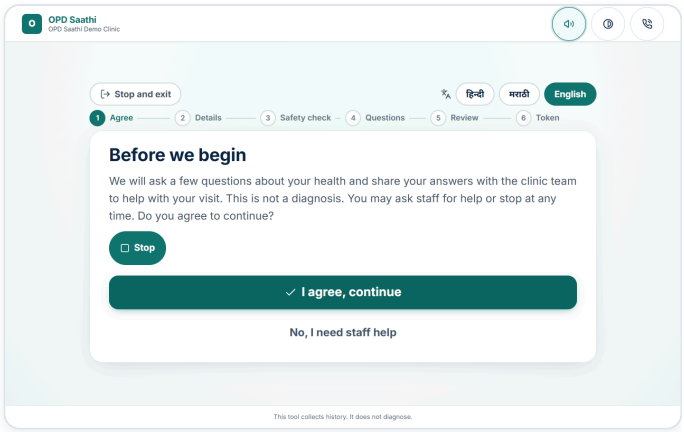
4 · END-TO-END WORKFLOW

Sunita's visit — 9:12 to 9:16 at the kiosk

Sunita, 34, has had fever and cough for three days. She speaks Marathi and reads little English. Follow her through the system — and see what happens behind each screen.



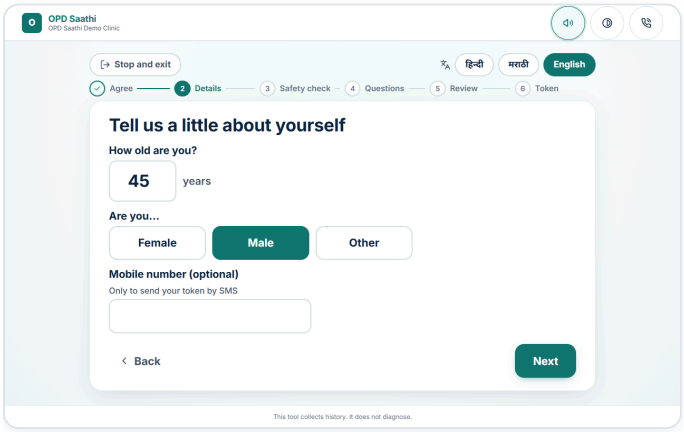
9:12 — The kiosk shows the hospital's name. She taps मराठी (Marathi), then Start.



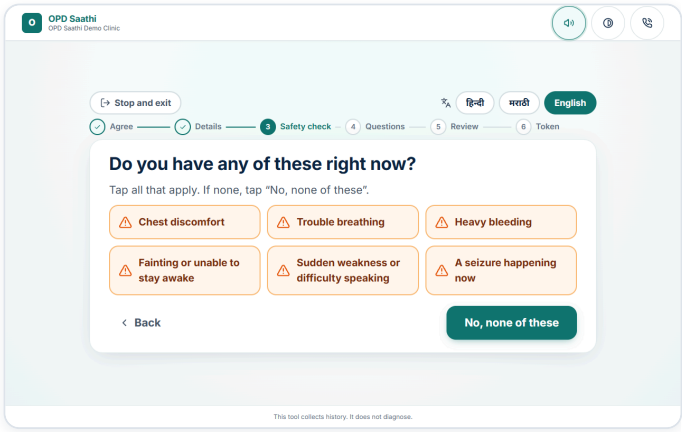
9:12 — Consent is read aloud (shown here in English). "No, I need staff help" is always there.

BEHIND THE SCENES

The kiosk is a browser that was set up once with a one-time activation link and keeps itself signed in with its own device key. Pressing Start asks the **FastAPI** server to open a visit; the server checks the kiosk is still in service, creates the visit in **PostgreSQL** and a short-lived session in **Redis**. The consent text is versioned and hashed, so the exact words she agreed to are recorded. The screen is read aloud by **Sarvam Bulbul** (natural Indian voices), or by the computer's own voice when there is no internet.



9:13 — Only age and sex are required; a phone number is optional.



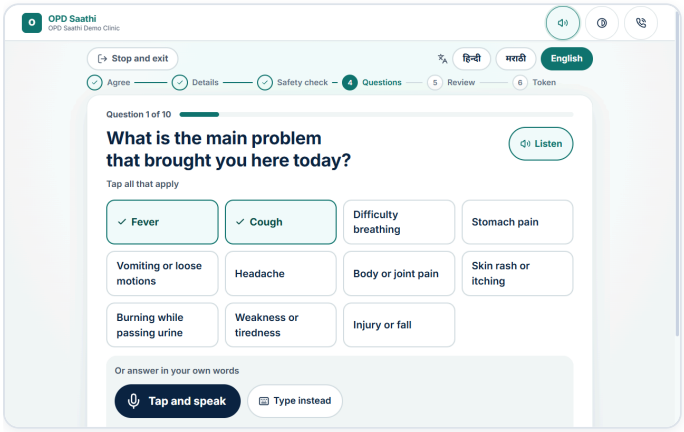
9:13 — Six danger-sign tiles come before any question. She taps "None of these".

BEHIND THE SCENES

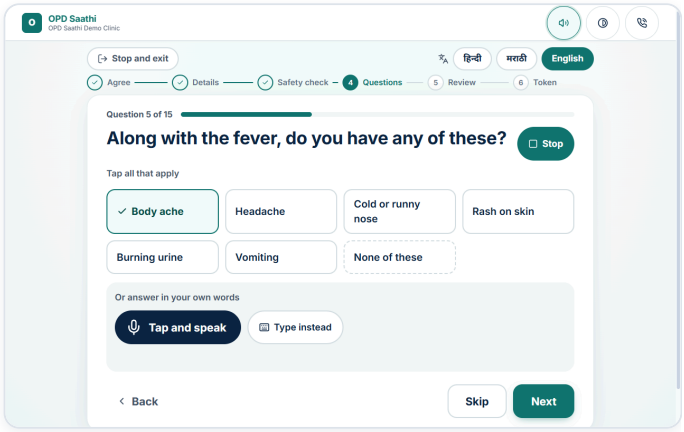
Had she tapped "Chest discomfort", the server would raise an **urgent red flag**, write it to the database and publish it instantly to every nurse's screen over a **WebSocket** — before a single history question is asked. Names and phone numbers are encrypted with **AES-256-GCM** before they are stored.

4 · END-TO-END WORKFLOW (CONTINUED)

Questions that adapt, and a history she confirms



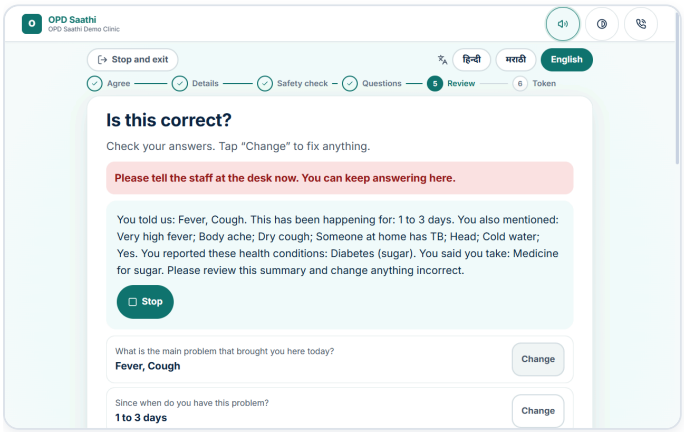
9:14 — "What is troubling you?" She taps Fever and Cough, or taps the mic and speaks.



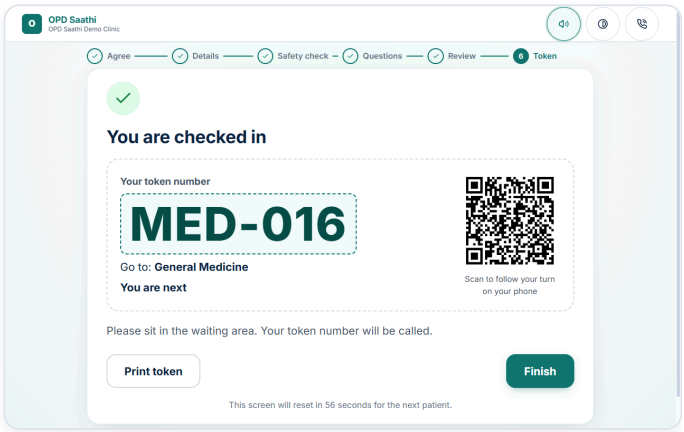
9:14 — The next questions follow her answer: fever pattern, cough, breathing.

BEHIND THE SCENES

The **interview engine** builds 8–15 questions from 13 topic modules (fever, breathing, stomach, pain, urine, skin, injury, sleep, past illness, medicines, allergies, family, other) and skips what does not apply — no pregnancy questions for men or children. When she speaks, her browser streams 16 kHz audio over a WebSocket to the server, which relays it to **Sarvam Saaras** (real-time Indian-language speech recognition); her words appear on screen for her to check. With a key, **Google Gemini** picks the most useful next question and may add one short follow-up — always validated against the question bank. **15 safety rules** scan every answer in every language: "blood in phlegm" would raise a flag immediately.



9:15 — The whole history is read back in her language (shown here in English). "Change" beside every answer.



9:16 — Token, department, place in line and a QR code. The screen resets for the next patient.

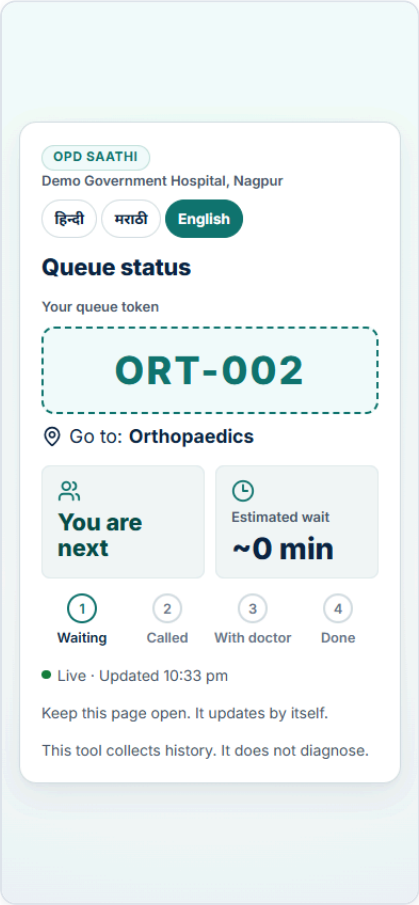
BEHIND THE SCENES

Confirming runs four things at once: the **routing engine** picks the department from her complaint (keyword rules in all three languages, with paediatric routing under 14); the **triage scorer** computes her 0–100 score; a **case sheet** is assembled in English with her original Marathi kept beside each field; and the visit is placed in the department's queue — a **Redis sorted set** updated by an atomic Lua script — with the next daily token number (MED-016).

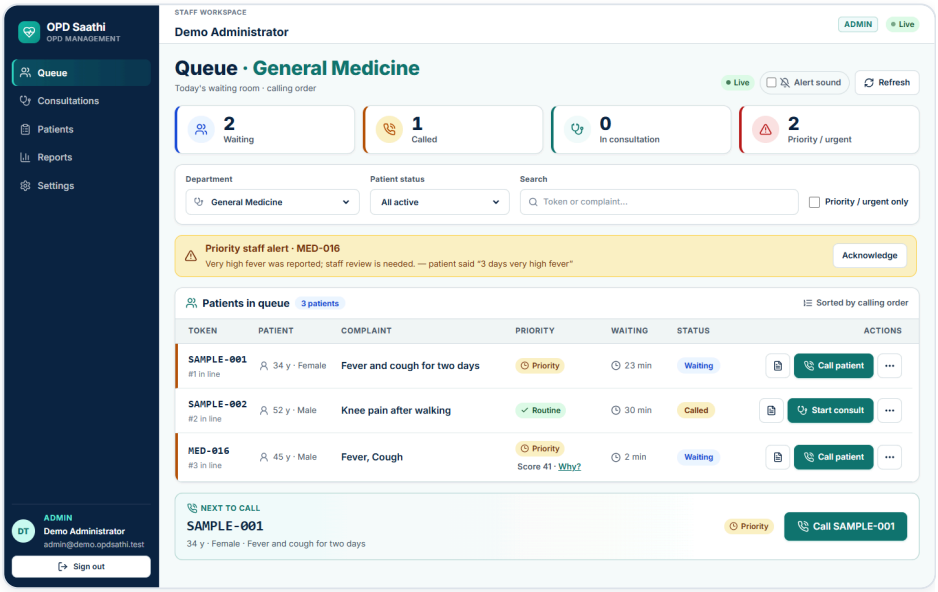
SAY IT ON STAGE

Sunita answered in Marathi, mostly by tapping, and heard her own history read back before confirming it. Four minutes later the doctor has it in English.

9:16 — Her phone, and the nurse's screen



She scans the QR: place in line, estimated wait, progress — in her language.



The nurse's live queue for General Medicine: counters, filters, alerts, fair calling order and "Next to call".

BEHIND THE SCENES

The phone page holds an open **Server-Sent Events** stream, so it updates by itself the moment the queue moves — no refresh, no app, no login, and no personal or health details on the page. The nurse's screen uses a **WebSocket**: when any visit changes, the server writes the change to an **outbox table** in the same database transaction, the **ARQ worker** relays it through **Redis pub/sub** within about two seconds, and every open staff screen updates. If Redis were ever lost, the whole queue is rebuilt from PostgreSQL. The estimated wait is a five-minute range (for example 15–20 min), calculated from the last 20 consultation times and the number of doctors working.

What the nurse can do

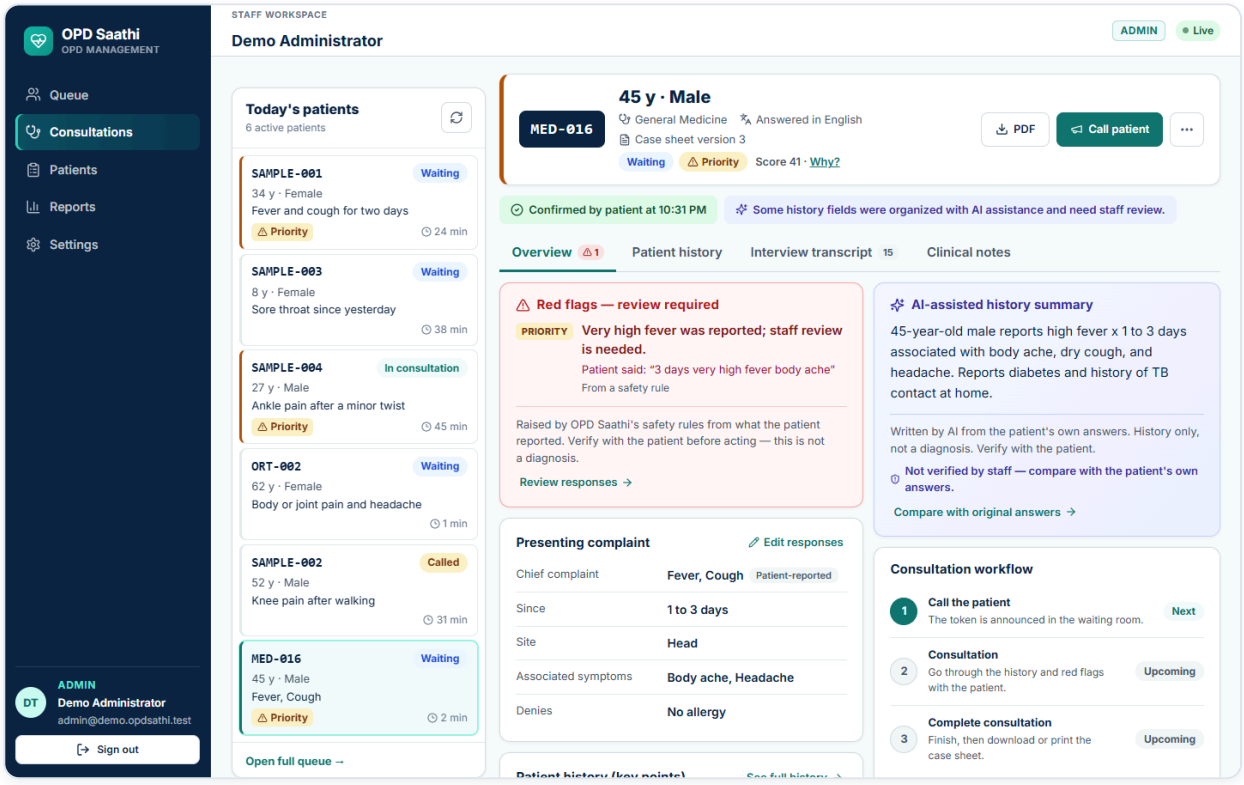
- Call the next patient, start and complete consultations, mark no-shows.
- Acknowledge alerts: red flags, "Patient asked for help", "AI suggests this patient may be more urgent — please confirm".
- Change priority — only with a written reason, kept in the audit log.
- Confirm the department when the routing was unsure; register a patient who needs help.

Built for a busy desk

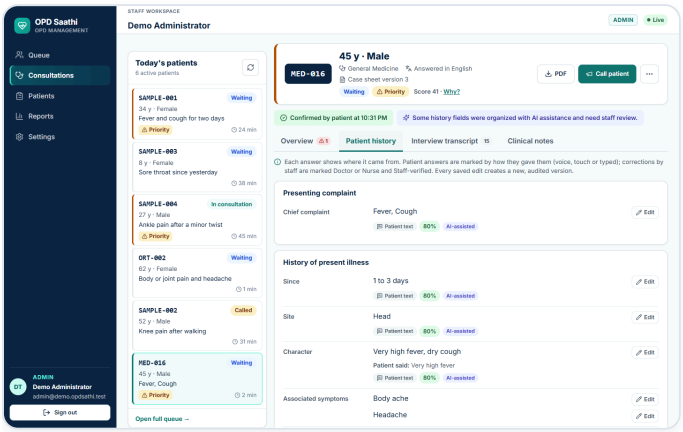
- Four counters (Waiting, Called, In consultation, Priority/urgent) double as filters.
- Search by token or complaint; optional alert sound; works on phones too.
- A 5,000-patient list applies an urgent change in about 22 ms (virtualised list).
- Unfinished visits from earlier days close automatically overnight.

4 · END-TO-END WORKFLOW (CONTINUED)

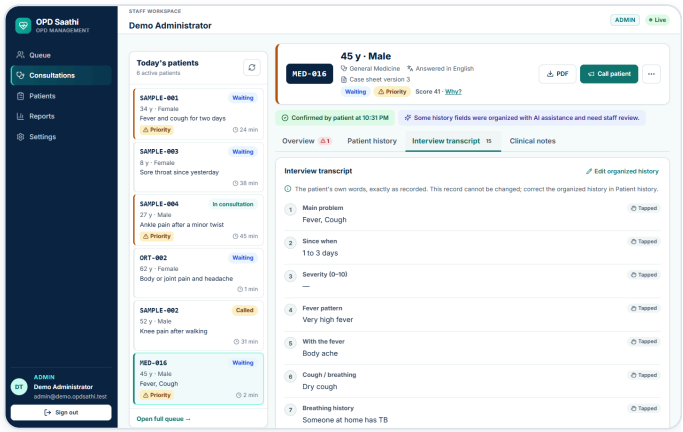
9:41 — The doctor's case sheet



Overview: red flags first, an AI-assisted summary clearly marked "not verified by staff", the complaint and the consultation workflow.



Full history, every field editable, with its source and the patient's own words.



The unchangeable interview transcript in the patient's own language, with how each answer was given.

BEHIND THE SCENES

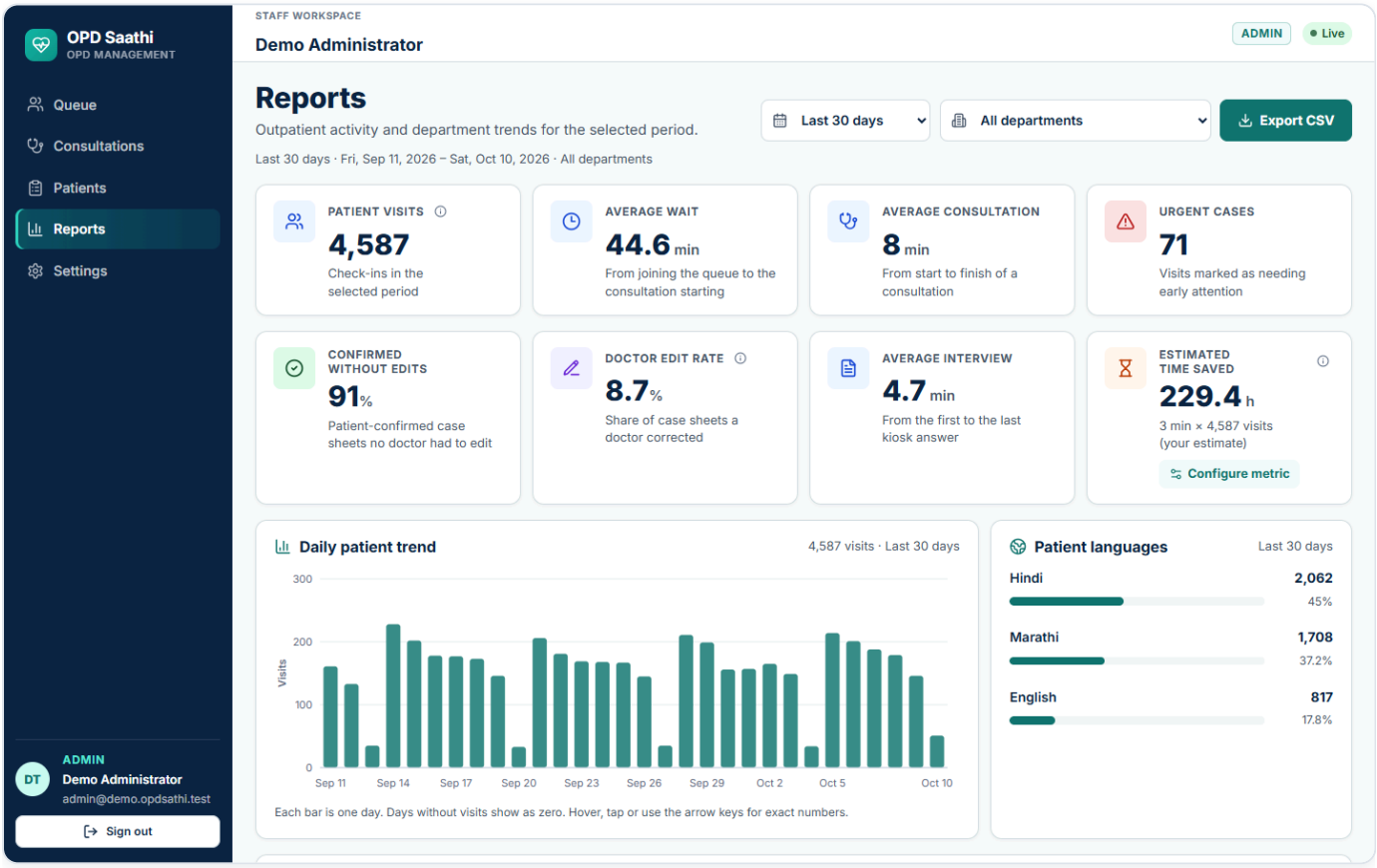
The case sheet is stored as validated JSON with a version number. Every doctor edit creates a new version marked **Staff-verified**; if two people edit at once, the second is told instead of silently overwriting (optimistic concurrency with ETags). The **PDF** is rendered on the server by **WeasyPrint** with Indian-language fonts. The AI summary is produced by **Gemini** only from her answers, passed through an **output guard** that rejects any diagnosis, treatment advice, dose or link — and it stays labelled unverified until a person checks it.

SAY IT ON STAGE

The doctor opens one page: urgent warnings first, an English summary, and Sunita's own Marathi words one click away. Every correction is versioned and audited.

4 · END-TO-END WORKFLOW (CONTINUED)

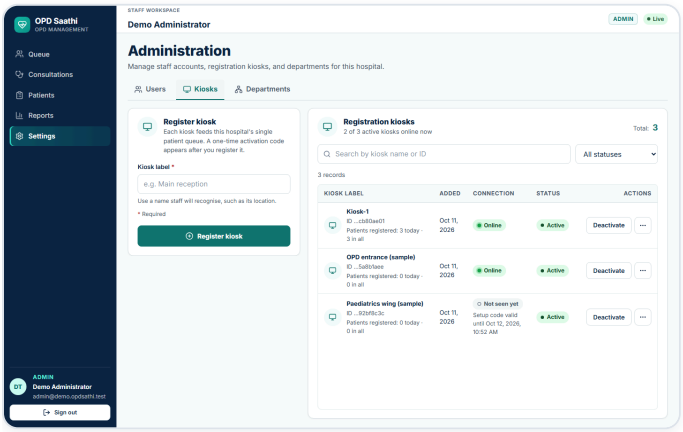
End of day — reports and administration



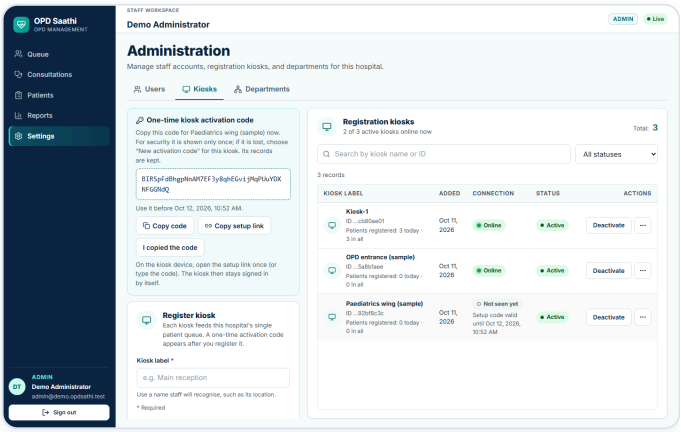
Reports for the last 30 days of the demo hospital: visits, average wait, consultation time, urgent cases, edit rate, interview time, estimated time saved, daily trend, languages and department load.

BEHIND THE SCENES

Reports read from a pre-computed PostgreSQL **materialized view** that the worker refreshes every five minutes, so even 100,000 visits summarise in about 2 ms. Filters for period and department drive every card, the chart and the CSV export together. "Estimated time saved" is clearly labelled as an estimate (visits × the minutes the hospital assumes each kiosk interview saves).



Settings → Kiosks: live connection, patients registered per kiosk, a kiosk waiting for setup.



A one-time activation code with Copy code and Copy setup link; valid for 24 hours.

The administrator adds doctors, nurses and departments, registers each kiosk and sets it up by opening one link on the device. Kiosks report in every minute; a replaced device gets a new code for the same kiosk, keeping its history. Staff, kiosks and departments can be turned off or deleted — deleting never erases medical history.

5 · EVERY FEATURE

The complete feature list

Patient kiosk

- Hindi, Marathi, English; switch language on any screen.
- Consent first, read aloud, versioned and recorded.
- Age and sex only; optional phone and SMS opt-in.
- Six danger-sign tiles before any question → instant urgent alert.
- 8–15 adaptive questions from 13 modules; one tap, speak, or type.
- Live speech-to-text shown for checking; Listen button on every question.
- Photo of an old prescription → medicines read automatically (AI).
- Back, Stop and exit, idle reset, Call staff on every screen.
- Read-back in the patient's language with Change on every answer.
- Token, department, place in line, QR code, printable slip.
- High contrast, 64-px buttons, voice guidance toggle.
- Offline short form: encrypted on the device, synced later.
- Shows its own hospital's name; stays signed in by itself.

Patient phone

- QR status page — no app, no login, no personal details.
- Live position, wait range and 4-step progress, in their language.
- Optional SMS / WhatsApp updates (provider ready, off by default).

Nurse

- Live fair queue per department with counters, filters and search.
- Real-time alerts with the patient's own words; acknowledge; alert sound.
- Call, start, complete, no-show; "Next to call" panel.
- Priority override with mandatory reason; department confirmation.
- "Why?" breakdown of every score.
- Patients register: every visit by period, status, urgency, department.
- Assisted registration with the same consent and safety steps.

Doctor

- Four-tab case sheet: Overview, History, Transcript, Notes.
- Red flags first with evidence; AI brief marked unverified.
- Every field shows its source, confidence and original words.
- Edit with versioning, conflict protection and unsaved-change warning.
- PDF download and print; keyboard shortcuts.

Administrator and platform

- Reports: 8 cards, daily trend, languages, department load, CSV export.
- Hospital-wide "time saved" setting with who changed it.
- Users, kiosks, departments: create, turn off, delete (history kept).
- Kiosk setup links, new codes any time, online status, per-kiosk counts.
- One-command install per hospital; many hospitals fully isolated.
- Audit log of every important action.
- Installable PWA; responsive from 320 px phones to 1600 px desktops.

6 · THE QUEUING ALGORITHM

Fair, explainable, and impossible to game

A simple "first come, first served" line ignores the sick. A pure "sickest first" line can make a routine patient wait all day. OPD Saathi uses a **virtual arrival time**: each patient is placed as if they had arrived a little earlier — by an amount their triage score earns, with a hard limit.

Queue position = arrival time − head start · head start = 0.6 minutes × score, never more than 45 minutes · **urgent patients go to the front**.

Step 1 — the triage score (0–100, staff only)

- **Red flags**: urgent flag or danger tile 70; priority flag 30; +5 for each further flag (max +15).
- **Warning answers** tapped by the patient: blood in phlegm +25, blood in urine +12, breathless +10, very little urine +10… all together at most 25.
- **Vulnerability**: infant under 1 +20, child 1–4 +10, aged 75+ +12, 65–74 +8, pregnant +12.
- **Onset**: started today +6, 1–3 days +3.
- **Self-rating** 7–10/10: only +1 to +4, and nothing for problems lasting months.

Step 2 — level and head start

- Urgent any urgent red flag or score ≥ 70 → front of the queue, in arrival order.
- Priority score 30–69 → head start 18–41 minutes.
- Routine score below 30 → head start 0–17 minutes.
- The level bands guarantee a nurse's override always wins over the points.
- **AI is a checker**: an AI flag counts as priority at most, can only raise urgency, and must quote the patient.

A worked example — General Medicine, one morning

PATIENT	ARRIVED	WHAT THE KIOSK RECORDED	SCORE · LEVEL	HEAD START	VIRTUAL TIME	CALLED
Mr Rao, 68	9:40	Tapped "Chest discomfort"	urgent	front	—	**1st**
Meena, 26	9:20	Pregnant (+12) + priority red flag (+30)	42 · priority	25 min	8:55	**2nd**
Ravi, 30	9:05	Fever started today (+6), shivering (+4)	10 · routine	6 min	8:59	**3rd**
Ramesh, 45	9:00	Knee pain for 2 weeks	0 · routine	0	9:00	**4th**
Baby Aarav, 8 months	9:30	Infant (+20), fever today (+6)	26 · routine	16 min	9:14	**5th**

No one waits forever

Ramesh can only be passed by someone who arrived less than 45 minutes after him, or by an urgent case. Everyone else stays behind him.

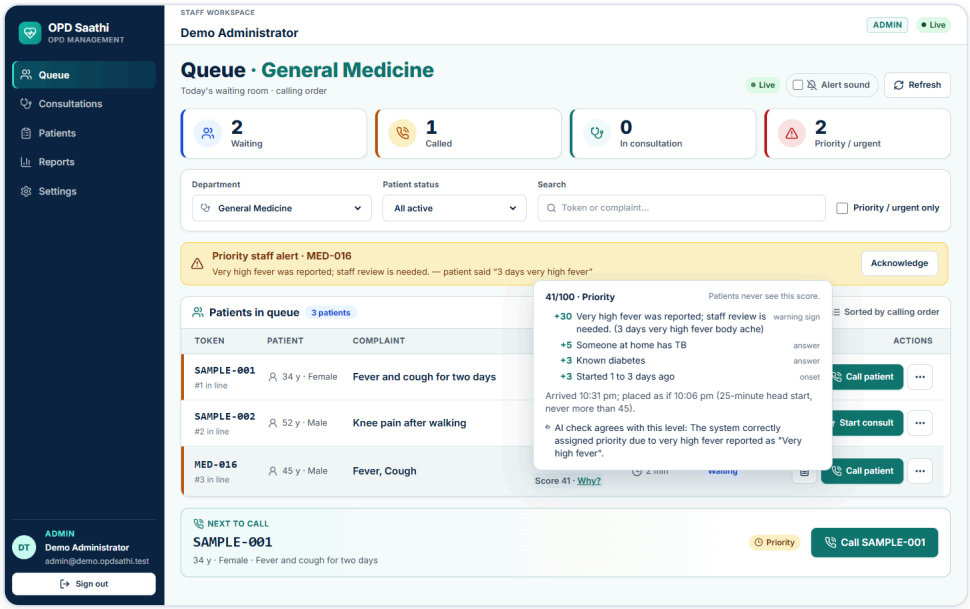
Can't be gamed

Typing "10/10" with no other signs earns 4 points — a 2-minute head start. Real warning signs, age and pregnancy are what count.

Fully explainable

"Why?" shows every line above to staff. Weights live in one readable file marked for clinician review.

Explainable on screen, fast in memory



"Why?" on a queue row: the score, the level, every point with its reason, the head start it earned, and the AI check.

Estimated wait, honestly

- Average consultation = mean of the last 20 consultations in that department (default 4 minutes until five exist), bounded to 1–15 minutes.
- Baseline = (patients ahead) × average ÷ doctors working.
- Shown as a range — −25 % to +25 %, rounded to 5 minutes — so we never promise false precision. Example: 4 ahead, 2 doctors, 8-minute consultations → "15–20 min".

How it runs

- Each department's queue is a **Redis sorted set** ordered by virtual time — insert and reorder in O(log n).
- Changes are applied by **atomic Lua scripts**, so two nurses clicking at once can never corrupt the order.
- **PostgreSQL** stays the source of truth; an **outbox** guarantees every change reaches Redis and the screens, and the queue can be rebuilt from the database at any time.
- Urgent changes reach staff screens within about two seconds; a 5,000-row list re-renders in ~22 ms.

SAY IT ON STAGE

Our queue is fair and explainable: urgent first, a bounded head start for the vulnerable and the unwell, a hard 45-minute cap so no one waits forever, a self-rating that can't cheat — and a "Why?" for every position.

7 · AI AND VOICE

AI that assists, never decides

AI makes OPD Saathi smarter in the patient's language — but every core feature works without it, and it is never allowed to diagnose, advise, or lower urgency.

SERVICE	WHAT IT DOES IN OPD SAATHI	WITHOUT IT
Sarvam Saaras real-time speech-to-text (saaras v3 realtime; v4 for clips)	Turns spoken Hindi, Marathi and English answers into text while the patient speaks, keeping the original script. Built in India for Indian languages and accents.	The browser's own speech recognition (Chrome/Edge), or tap/type.
Sarvam Bulbul text-to-speech (bulbul v3)	Reads every screen, question and the patient's read-back aloud in natural Indian voices.	The computer's built-in voice.
Google Gemini Flash-Lite models, structured JSON output	Translates free-text answers to English; writes the doctor's 2–3 line brief; chooses the most useful next question and short follow-ups; reads old prescription photos; gives a second opinion on warning signs; reviews the triage after check-in.	Question bank order, bilingual answer bank for translation, safety rules only.

Guard-rails on every AI call

- Names and identifiers are **removed before** anything is sent; prompts and replies are not logged.
- Answers must match a strict JSON schema and are validated again on our side.
- An **output guard** rejects any text that diagnoses, advises treatment, gives a dose or contains a link.
- AI red flags must quote the patient's actual words, count as priority at most, and only ever raise urgency.
- 2.5-second deadline, one retry, a circuit breaker, and at most 30 calls per check-in — if AI is slow or down, the kiosk quietly continues with the question bank.

Why these providers

- **Sarvam** is an Indian AI company whose speech models are trained for Indian languages, accents and code-mixed speech — exactly our patients. Its real-time stream shows words as they are spoken.
- **Gemini Flash-Lite** is fast and low-cost, supports guaranteed JSON output, and handles Hindi and Marathi well — suited to many short tasks per patient.
- Every provider sits behind a small adapter, so a hospital can switch providers — or run with none — without touching the rest of the system.
- AI is switched on per installation with keys, and each hospital can opt out.

SAY IT ON STAGE

AI translates, summarises and double-checks — but rules decide, AI can only add caution, staff confirm, and the hospital can run the whole system with AI switched off.

Built for health data, layer by layer

LAYER	HOW OPD SAATHI PROTECTS IT
Data at rest	Names, phone numbers, answers and case sheets are encrypted field by field with **AES-256-GCM**, each value bound to its own row and column so it cannot be copied elsewhere. Phone lookups use a keyed **HMAC-SHA256** digest, never the number itself.
Hospital isolation	**PostgreSQL row-level security** is forced on all 17 tables: the database itself refuses to return another hospital's rows, even if the application had a bug. Verified live: a second hospital could not read, call, change or delete anything of the first.
Staff sign-in	Passwords hashed with **Argon2id**. Signed 10-minute access tokens (JWT) kept only in memory; a rotating refresh cookie that is httpOnly and SameSite=Strict, with reuse detection that revokes the whole session. Five wrong passwords lock the account for 15 minutes.
Kiosks	One-time activation codes that expire after 24 hours and are stored only as hashes; each kiosk then holds its own device key. Turning a kiosk off, deleting it or re-pairing it cuts the old device off immediately.
Access control	Five roles (admin, doctor, nurse, kiosk, patient); every one of the 55 private API endpoints checks the role; a patient token sees only its own visit; nobody can lock themselves out.
Abuse and mistakes	Rate limits on sign-in and kiosk activation (Redis sliding window); every change carries an **idempotency key**, so a double tap or retried request never creates two visits; strict input validation and request-size limits; CSV exports protected from spreadsheet formula tricks.
Browser	A strict **Content-Security-Policy**, no framing, no-sniff and no-referrer headers; staff tokens never stored in the browser; the offline form is encrypted on the device with a key that cannot be exported.
Accountability	An append-only **audit log** records sign-ins, edits, overrides (with reasons), kiosk activations and admin changes. Deleting a record archives it, so medical history is never lost.
Secrets	Each installation generates its own passwords, encryption and signing keys; production start-up refuses weak or default keys.
Every change, checked	GitHub Actions run **bandit** and **semgrep** code scanning, **pip-audit** and **npm audit** dependency checks and **gitleaks** secret scanning; code cannot reach the main branch unless all pass.

Privacy by design

Only age and sex are required. The phone status page shows no name or health details. Patient data stays on the hospital's own server. An optional retention tool removes old transcripts and pseudonymises patients.

Clinical safety by design

No diagnosis, disease names, medicines or advice anywhere. Danger signs first. Uncertainty always escalates. The patient confirms the read-back; doctors can correct anything.

9 · SCALABILITY AND RELIABILITY

From one kiosk to many hospitals

Many kiosks, one queue

Any number of kiosks per hospital feed the same department queues. Tokens are numbered per department per day, whichever kiosk was used. Each kiosk shows its hospital's name and reports online/offline every minute.

Many hospitals, fully separate

Each hospital gets its own installation and database — set up with one command. Several hospitals can also share one server (for example a district hosting its PHCs), separated by the database's row-level security.

Built to scale out

- The API is stateless: run more copies behind a load balancer.
- Queues in Redis sorted sets (O(log n)); reports from a pre-computed view.
- Virtualised lists keep staff screens fast with thousands of rows.
- Two Redis instances: durable state (append-only file) and a separate, size-bounded cache.

Built not to break

- **Outbox pattern**: queue changes are written in the same transaction as the visit, then delivered — nothing is lost if a service restarts.
- The live queue can be rebuilt from PostgreSQL at any time.
- Idempotency keys stop duplicate visits from retries.
- 5-second database statement timeout; services restart automatically.
- Offline kiosk form when the network fails.

Works where the internet does not

SITUATION	WHAT HAPPENS
Internet down, hospital network up	Everything keeps working — check-in, queue, case sheets, reports — because kiosks talk to the hospital's own server. Only AI, natural voices and voice typing pause; patients tap or type.
Kiosk cannot reach the server	The app is cached on the kiosk (PWA service worker), so it still opens. Within about 10 seconds it offers a short form, encrypted on the device (AES-256-GCM, IndexedDB), with a provisional token. It syncs automatically when the link returns — never creating a duplicate.
Slow networks	The app is downloaded once and cached; afterwards only small JSON messages travel. Kiosks stay signed in through outages.

Measured

- Reports summary over 100,000 visits: 1.88 ms (95th percentile) on a laptop.
- 5,000-patient queue: urgent update applied in about 22 ms.
- Lighthouse accessibility 100 on kiosk and status pages.

> Honest note: these are laptop measurements and a small load smoke test, not a certified capacity figure. The path to larger scale (connection pooling, read replicas, managed database) is documented.

Designed for people under pressure

For patients

- One question per screen, big tappable answers (minimum 64 × 64 px).
- A six-step progress bar on every screen: Agree → Details → Safety check → Questions → Review → Token.
- Everything read aloud; high-contrast mode; plain, neutral wording.
- Mistakes are easy to undo: Back, Change, Stop and exit.

For staff

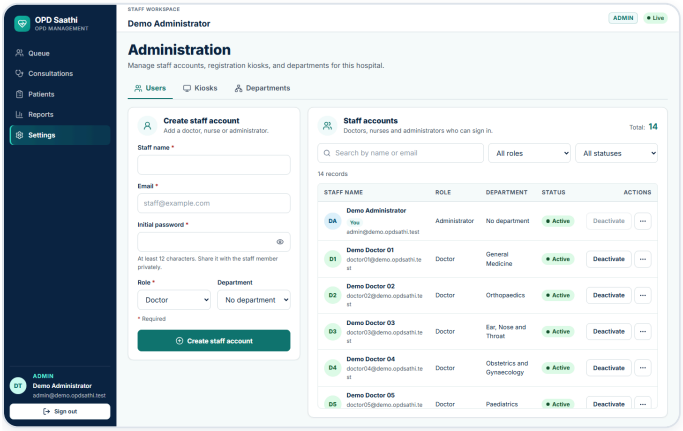
- What matters first: red flags at the top, one main action per patient.
- Same layout on laptop, tablet and phone; never scrolls sideways (checked at 7 widths from 320 to 1600 px).
- Clear status colours, counters that filter, and confirmation before anything risky.

Accessible and inclusive

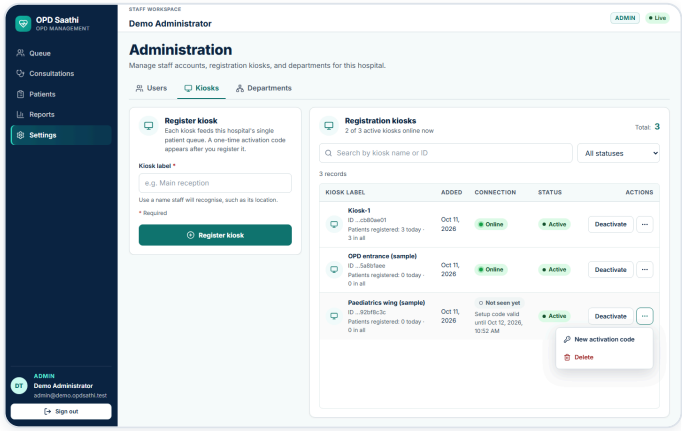
- Zero serious accessibility issues in automated (axe) scans.
- All colour pairs pass WCAG AA contrast — checked in CI.
- About 830 translated phrases per language, kept in sync by a CI check.

Clean engineering

- Clear modules: interview engine, safety rules, routing, triage, queue, case sheets — each testable on its own.
- Every AI and voice provider behind a small adapter.
- Typed end to end: the API's schema generates the web app's TypeScript types.
- Rules and weights in readable files for clinicians to review.



Administration keeps forms on the left and the list on the right; headings stay put while lists scroll.



Rare actions live in a ⋯ menu; risky ones always ask first.

The tech stack — what each part does, and why

Front end (what everyone sees)

TECHNOLOGY	ITS JOB IN OPD SAATHI	WHY WE CHOSE IT
React 18 + TypeScript	All screens: kiosk, phone status page, staff workspace.	One component-based app for every audience; TypeScript catches mistakes before they reach a patient.
Vite	Builds and serves the web app.	Very fast builds and a small, split production bundle for slow kiosks.
PWA (vite-plugin-pwa, Workbox)	Caches the app on kiosks; offline short form.	Kiosks keep opening without a network; installs like an app with no app store.
IndexedDB + Web Crypto	Stores offline check-ins encrypted on the kiosk.	Built into every browser; AES-GCM with a key that cannot be exported.
i18next	Hindi, Marathi and English text everywhere.	Mature translation library; adding a language means adding one file.
TanStack Query & Virtual	Data fetching/caching; very long queue lists.	Keeps screens fresh without reloads; renders only visible rows of a 5,000-row list.
Zustand	Small shared state (language, kiosk preferences).	Tiny and simple compared to heavier state libraries.
Web Speech API	Browser voice when no AI keys are set.	Free fallback built into Chrome/Edge.

Back end (the brain)

TECHNOLOGY	ITS JOB IN OPD SAATHI	WHY WE CHOSE IT
Python 3.12 + FastAPI	The API: interview engine, safety rules, triage, routing, queue, case sheets, reports, auth.	Async and fast; first-class WebSockets; Python suits rules and AI work; automatic API documentation.
Pydantic	Validates every request, case sheet and AI reply.	Strict schemas reject bad or unexpected data before it is stored.
SQLAlchemy 2 + asyncpg	Talks to PostgreSQL.	Non-blocking database access with parameterised queries (no SQL injection).
Alembic	Database migrations (16 so far).	Every hospital's database upgrades safely and repeatably.
WebSockets & Server-Sent Events	Live staff queue and alerts; patient phone status; streaming voice.	Instant updates without page refresh; SSE is light enough for any phone.
ARQ worker	Relays queue changes, refreshes reports every 5 min, closes old days.	Async background jobs on the Redis we already run — no extra broker.
WeasyPrint	Case sheet PDFs.	Renders HTML to print-quality PDF with Hindi/Marathi fonts, on the server.
Argon2, PyJWT, cryptography	Password hashing, sign-in tokens, field encryption.	Current, well-reviewed standards: Argon2id, HS256 JWT, AES-256-GCM.
Gunicorn + Uvicorn	Runs the API with several worker processes.	Production-grade process manager for async Python.

Data, AI, infrastructure and quality

TECHNOLOGY	ITS JOB IN OPD SAATHI	WHY WE CHOSE IT
PostgreSQL 16	The source of truth: hospitals, staff, kiosks, visits, case sheets, audit log; report views.	Reliable relational database with **row-level security** for hospital isolation, JSON columns for case sheets, and materialized views for fast reports.
Redis 7 (state)	Live queues (sorted sets + Lua), kiosk sessions, rate limits, idempotency, pub/sub for live updates.	In-memory speed for the queue; atomic scripts so concurrent clicks cannot corrupt order; durable append-only file.
Redis 7 (cache)	Short-lived AI response cache.	Kept separate and size-bounded, so cache pressure never touches the queue.
Sarvam Saaras / Bulbul	Indian-language speech-to-text and text-to-speech.	Built for Indian languages and accents; real-time streaming.
Google Gemini (google-genai)	Translation, summary, next question, prescription reading, second opinion.	Fast, low-cost Flash-Lite models with guaranteed JSON output.
Docker + Docker Compose	Packages the database, both Redis instances, API, worker and web server.	The same system runs identically on any laptop or hospital server; one command to start; services restart by themselves.
nginx	Serves the web app and forwards `/api` to the server.	Fast static hosting, one address for kiosks and phones, security headers.
PowerShell setup scripts	`start-demo.ps1` and `setup-hospital.ps1`.	Hospital IT can install and update with one command on Windows.
Prometheus + Grafana (optional)	Request metrics and dashboards.	Standard open-source monitoring when a hospital wants it.
pytest, Vitest, Playwright, axe	814 server tests, 54 web tests, 132 browser tests, accessibility scans.	Every feature is checked automatically in a real browser, in three languages.
ruff, mypy (strict), ESLint	Code quality and type checking.	Catches bugs before tests even run.
GitHub Actions	Runs every check on every change; protects the main branch.	Nothing reaches main unless backend, frontend and security jobs pass.
bandit, semgrep, pip-audit, npm audit, gitleaks	Security scanning of code, dependencies and secrets.	Security is checked automatically, not occasionally.
k6	Load smoke tests.	Simple scripted load testing.

SAY IT ON STAGE

A modern, open, mostly free stack: React for every screen, Python FastAPI for the rules, PostgreSQL for safe records, Redis for a live queue, Sarvam and Gemini for Indian-language AI — all packaged in Docker and started with one command.

12 · DEPLOYMENT

How a hospital gets OPD Saathi

Each hospital gets its own installation on a PC or server inside the hospital. Patient data never leaves the building, and the OPD keeps running when the internet does not.

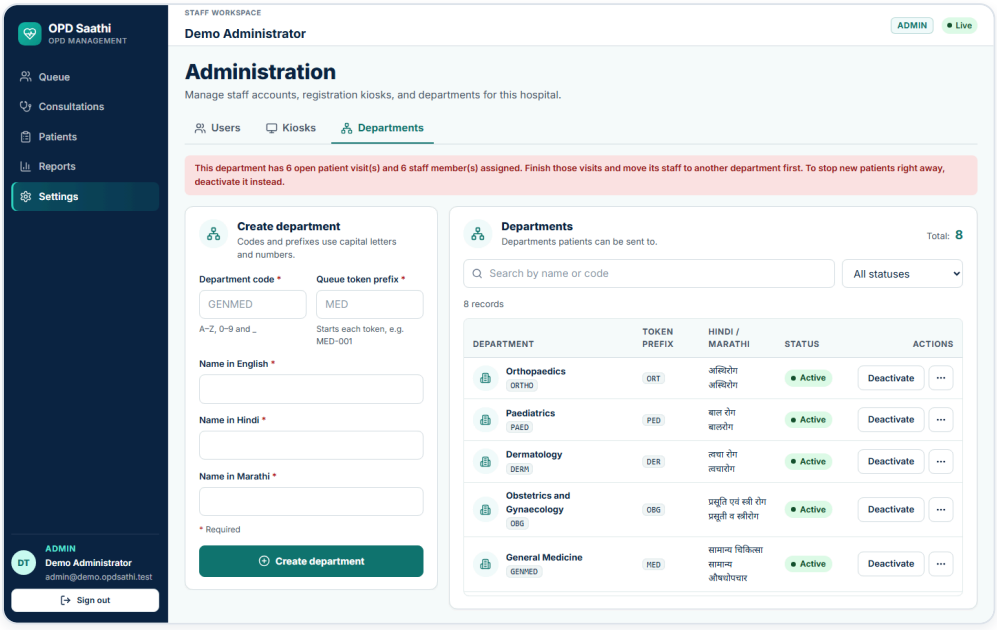
1. **Prepare a server** on the hospital network with Docker; give it a fixed local address.
2. **Run one command** — `.\setup-hospital.ps1`. It creates fresh secrets and encryption keys, builds and starts the database, Redis, API, worker and web app, and asks for the hospital's name and first administrator. It also creates the 8 standard departments.
3. **The administrator signs in** and adds doctors, nurses and departments in Settings.
4. **Each kiosk is set up** by registering it and opening its setup link once on the device; it then stays signed in and appears as Online.
5. **Optional**: add Gemini and Sarvam keys for AI and natural voices; set up HTTPS for voice on every kiosk; schedule backups.

One installation per hospital (recommended)

Own server, own database, own keys. Best for privacy and for hospitals with unreliable internet.

Several hospitals on one server

For example a district hosting its PHCs. Each hospital is added with one command and separated by row-level security.



Administration protects the hospital from mistakes: a department with open visits or staff cannot be deleted.

13 · EVIDENCE

Proof that it works

814
server tests
plus database integration tests for sign-in, kiosks, isolation and offline replay

132
browser tests
real screens, 3 languages, 7 screen widths, accessibility scans

54
web unit tests
queue, case sheet, reports, settings, kiosk

Synthetic evaluations

CHECK	RESULT
Interview scenarios completed correctly	30 / 30
Department routing test cases	96 / 96
Red flags that must be caught	60 / 60
Negations not falsely flagged ("no chest pain")	50 / 50
History extraction with zero invented fields	100 / 100
Adversarial prompts blocked by the AI output guard	100 / 100
Urgency never decreases when a warning sign is added	500 / 500
Second hospital fully isolated from the first (live check)	20 / 20
Kiosk set-up, renewal, re-pairing and delete rules (live check)	35 / 35

Quality gates on every change

Backend, frontend and security jobs in GitHub Actions — tests, type checks, migrations up and down, accessibility, contrast, translations, generated API types, code and dependency scanning, secret scanning. The main branch only accepts changes that pass all of them.

> These are synthetic tests: they prove the software behaves as designed. Clinical accuracy still needs review by clinicians and native speakers.

What changes on day one

BEFORE	WITH OPD SAATHI	WHO BENEFITS
History taken by the doctor, question by question	History ready before the consultation, in English, with the patient's own words	Doctors get time back for examination and care
Patients struggle in a second language	Check-in in Hindi, Marathi or English, by tap or voice	Patients who cannot read or speak the doctor's language
Serious patients wait unnoticed	Danger signs first; urgent alert to staff in seconds	The seriously ill
"Why were they called first?"	A fair, explainable queue with "Why?" on every position	Everyone in the waiting room; staff
Crowding at the doctor's door	A live token on the patient's own phone	Patients, families, nurses
No data to plan staffing	Reports on waits, load, languages and time saved	Administrators and health officials

Time saved — an estimate

Reports multiply visits by the minutes of history-taking each kiosk interview is assumed to save. With the demo's assumption of 3 minutes, 4,580 visits in 30 days come to about 229 doctor-hours. Each hospital sets its own assumption, and the screen labels it as an estimate.

Where it goes next

1. Clinician and native-speaker review of all questions, rules and weights.
2. A pilot in one OPD department with real kiosks and staff feedback.
3. Doctor's notes and e-prescription fields; SMS provider connected.
4. ABHA / ABDM and hospital information system integration.
5. More Indian languages — the app is built for it.

> **SAY IT ON STAGE**
>
> OPD Saathi gives doctors back the minutes spent collecting history, gives patients a voice in their own language, and gives everyone a queue they can trust.

15 · HONEST STATUS

What is not done yet

Being clear about limits makes the story stronger, not weaker.

ITEM	STATUS
Clinical review	Questions, safety rules, triage weights and consent text are drafts awaiting clinician and native-speaker sign-off.
Doctor's notes	Examination findings and plan are recorded in the hospital's own system for now.
HTTPS on hospital networks	Needed for voice on kiosks other than the server itself; tapping and typing work everywhere today.
A few admin screens	Editing a staff member's details, resetting a password and switching AI on/off per hospital are supported by the server but have no screen yet.
Integrations	No ABHA/ABDM, FHIR or hospital-system connection yet; SMS needs a provider account.
Real-world testing	Load numbers are laptop measurements; first-time patients still need to be observed using the kiosk without coaching.

16 · FOR THE PRESENTATION

A 10-minute talk and live demo

TIME	SECTION	SAY / SHOW
0:00	The need	The OPD corridor: languages, literacy, minutes lost to history-taking, an invisible queue. (Chapter 1)
1:00	Who we serve	Elderly, non-readers, Hindi/Marathi speakers, mothers and infants, the seriously ill. (Chapter 2)
1:45	Live demo: kiosk	Tap मराठी → consent read aloud → age/sex → "None of these" → tap Fever + Cough, speak one answer → read-back → token + QR. Scan the QR with a phone.
4:00	Live demo: staff	New token in the queue in English; click "Why?"; open the case sheet; show the transcript tab.
5:00	The urgent moment	New check-in → tap "Chest discomfort" → the urgent alert appears instantly on the staff screen.
5:45	The fair queue	Virtual arrival time, the 45-minute cap, "10/10 can't cheat" — the worked example. (Chapter 6)
6:45	AI that assists	Sarvam for Indian-language voice, Gemini for translation and summaries, guard-rails, works without AI. (Chapter 7)
7:30	Secure and scalable	Encryption, row-level security, many kiosks one queue, one install per hospital, works offline. (Chapters 8–9, 12)
8:30	Tech and proof	The stack in one line; 1,000+ automated checks; evaluation scores. (Chapters 11, 13)
9:15	Impact and next steps	Time back for doctors, voice for patients, a queue to trust; clinician review and a pilot next. (Chapter 14)

Before you start

- Run `.\start-demo.ps1 -NoBuild`; kiosk full-screen (F11) in one window, staff workspace signed in as doctor01 or admin in another.
- Laptop and phone on your own hotspot (for the QR scan and for AI/voice).
- If the internet fails, tap instead of speaking — everything else still works.

Questions the judges may ask

QUESTION	ANSWER
Does it diagnose?	No — by design. It collects history and flags warning signs for staff. An output guard blocks any AI text that diagnoses or advises treatment.
What if the AI is wrong?	AI can only add caution: its flags count as priority at most, must quote the patient, and staff confirm. Safety rules decide. The AI summary is marked unverified until a person checks it.
What if there is no internet?	Kiosks talk to the hospital's own server, so check-in and the queue keep working. If even the local network fails, the kiosk switches to an encrypted offline form that syncs later.
Can someone jump the queue?	No. The self-rating is worth at most 4 of 100 points; priority comes from real signs, age and pregnancy; the head start is capped at 45 minutes; overrides need a written reason and are audited.
How is patient data protected?	Field-level AES-256-GCM encryption, per-hospital row-level security in the database, Argon2id passwords, short-lived tokens, an audit log, and data that stays on the hospital's own server.
How does it scale?	Any number of kiosks feed one queue per department; queues live in Redis sorted sets; the API is stateless and can be replicated; each hospital is a separate installation set up with one command.
What does it cost to run?	The stack is open-source and runs on an ordinary PC. Without AI there is no per-patient cost; AI adds small per-call costs on fast, low-cost models, and each hospital can switch it off.
What about people who cannot use a kiosk?	Voice, big buttons and Call staff help most; anyone else is registered by a nurse with the same consent and safety steps.
Why Sarvam and Gemini?	Sarvam's speech models are built for Indian languages and accents with real-time streaming; Gemini Flash-Lite is fast, low-cost and returns strict JSON. Both sit behind adapters and can be replaced.
Is it deployed?	It is designed to be installed per hospital on its own server — one command (`setup-hospital.ps1`). Today we show it on a laptop, exactly as it would run on a hospital PC.
What's next?	Clinician review, a one-department pilot, doctor's notes, ABHA/ABDM integration and more languages.

OPD Saathi collects history for clinical staff. It does not diagnose or recommend treatment. All patients, staff and hospitals shown in this document are fictional.